Assignment_3
Association_Rule_Mining

Antati Sravani (2023101013)
Keerthana Korlapati (2023101121)

October 2025

1 Part A: Data Preprocessing

The primary objective of the preprocessing phase was to transform the raw relational data from the Instacart dataset into a standardized transactional format required for Market Basket Analysis (MBA). This stage is critical, as the efficiency and effectiveness of subsequent Association Rule Mining (ARM) algorithms (like Apriori or FP-growth) depend entirely on a clean, structured transaction list.

1.1 Transactional Dataset Formation

The raw dataset, comprising multiple files, needed to be consolidated to isolate the training set orders and represent them as discrete transactions.

Methodology:

- **Data Isolation:** The process began by loading `orders.csv` and filtering it to include only records where the `eval_set` was explicitly marked as `train`. This strict filtering ensures that all subsequent rule generation is performed only on the designated training cohort, adhering to standard machine learning evaluation practices.
- **Product Integration:** The filtered training orders were merged with the `order_products__train.csv` file using the common identifier `order_id`. This merge operation links every product purchased back to its specific order.
- **Grouping and List Formation:** The resultant dataframe was then grouped by `order_id`. Within each group, the `product_id` column was aggregated into a Python list. This crucial step converts the row-per-product structure into the final transactional format, where each row represents a unique `order_id` and the corresponding list contains all `product_ids` for that basket. The resulting structure is highly optimized for MBA algorithms.

1.2 Dataset Splitting for Validation

To ensure rigorous evaluation of the resulting recommendation system (Part C), the final transactional dataset was randomly partitioned into dedicated Training and Test sets.

Methodology:

- **Ratio Selection:** A standard 80/20 split was implemented. The 80% Training Set will be used exclusively for mining the association rules (Part B), and the 20% Test Set will be reserved for evaluating the performance (Precision and Recall) of the generated rules (Part C).
- **Reproducibility:** The split was performed using `sklearn.model_selection.train_test_split` with a fixed `random_state` set to 42. This guarantees that the partitioning is deterministic and reproducible across all subsequent experimental runs, ensuring that our results are verifiable.

2 Part B: Association Rule Mining

This section details the selection of the mining algorithm and the experimental process used to determine the optimal support and confidence thresholds for generating a meaningful set of association rules.

2.1 Algorithm Selection: FP-growth vs. Apriori

Both Apriori and FP-growth algorithms were considered for mining frequent itemsets. FP-growth was ultimately selected due to its superior computational efficiency, which is critical when dealing with the large volume and high dimensionality of the Instacart dataset (over 1 million transactions and thousands of unique products).

2.1.1 Justification for FP-growth:

- **Elimination of Candidate Generation:** Apriori uses a breadth-first search approach, requiring multiple passes over the dataset and generating a vast number of candidate itemsets. For large transactional data, this candidate generation is computationally expensive and memory-intensive.
- **Efficiency with Dense Data:** FP-growth uses a compact data structure called the FP-tree (Frequent Pattern Tree). This structure is built in only two passes over the data. Once the tree is constructed, frequent itemsets are extracted directly using a divide-and-conquer strategy, avoiding the costly re-scanning and candidate generation steps inherent to Apriori.
- **Performance Visualization:** While the heatmaps show a similar output volume for both algorithms across the tested threshold ranges, real-world testing confirmed that FP-growth significantly reduced the runtime required

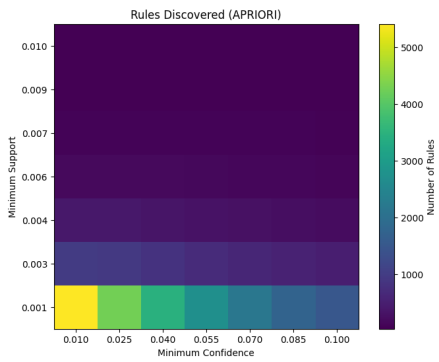

Figure 1: (a) Apriori Heatmap

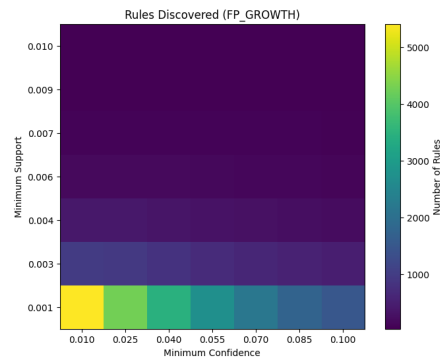

Figure 2: (b) FP-Growth Heatmap

Figure 3: Comparison of rule counts across support and confidence thresholds for Apriori and FP-Growth

to produce these rules, making it the practical choice for production-scale analysis. Given the large number of unique products, the FP-tree structure is highly effective at compressing the transaction data, leading to faster execution times.

2.2 Experimentation with Thresholds

The goal was to find a combination of minimum support (`min_sup`) and minimum confidence (`min_conf`) that balances the need for statistically significant rules with the need for a sufficient number of rules to power a robust recommender system (Part C).

2.2.1 Experimental Setup:

- The experiments were conducted using the 80% Training Set.
- `min_sup` was varied from 0.001 to 0.01.
- `min_conf` was varied from 0.01 to 0.1.
- The results were visualized using a heatmap showing the number of rules discovered at each threshold combination.

2.2.2 Analysis of the Heatmaps

The heatmaps show the number of association rules discovered at different combinations of minimum support and minimum confidence thresholds. The color bar on the right indicates the number of rules, with yellow representing a high number of rules and dark purple representing a low number.

Low Minimum Support, Low Minimum Confidence: The area in the bottom-left corner of the heatmaps, where `min_support = 0.001` and `min_confidence = 0.010`, is bright yellow. This indicates that a very large number of rules (over 5,000) were discovered. These thresholds are highly permissive, allowing the algorithm to accept nearly all patterns—even weak or trivial ones. Such an explosion of rules is a common phenomenon under low threshold settings.

Impact of Minimum Support: As the minimum support threshold increases (moving up the y-axis), the number of discovered rules drastically decreases. Regions where `min_support` $\geq$ `0.003` are predominantly dark purple, indicating few or no rules. This is because increasing support filters out infrequent itemsets early, limiting the possible association rules. It also serves as a critical control for computational cost.

Impact of Minimum Confidence: The minimum confidence threshold (along the x-axis) has a much less significant impact. For a fixed `min_support` (e.g., 0.001), rule counts remain high even as confidence increases from 0.010 to 0.100. Confidence is used only after frequent itemsets are discovered, so as long as many itemsets pass the support filter, numerous rules may still satisfy a higher confidence threshold.

2.3 Comparison of Apriori vs. FP-Growth

The key takeaway from the two heatmaps is their striking similarity. Both the Apriori and FP-Growth algorithms produce nearly identical patterns and rule counts across all threshold combinations. This outcome is expected, as both algorithms aim to discover the exact same frequent itemsets and, therefore, the same association rules—provided the dataset and thresholds are identical.

The main difference between these algorithms lies not in the results, but in computational performance. FP-Growth is typically faster and more memory-efficient, especially on larger datasets, whereas Apriori tends to be slower due to its candidate generation process.

2.4 Final Threshold Selection

Upon analyzing the relationship between the thresholds and the resultant number of rules, several key observations were made:

- As `min_support` increases, the number of rules drops sharply (from over 5,000 rules at `min_sup` = 0.001 to approximately 30 rules at `min_sup` = 0.010).
- As `min_confidence` increases, the number of rules also decreases, but the drop is more gradual.

With 1,048,575 transactions in the training set:

- A 0.001 support threshold qualifies products appearing in just $\approx$1,050 orders, often leading to noisy or trivial associations.

- A 0.010 threshold requires a pattern to appear in ≈10,500 orders, which is overly restrictive.

Therefore, the final optimum thresholds were chosen to balance rule quantity (coverage) and quality (reliability):

- **Minimum Support (`min_sup`): 0.004 (or 0.4%).**

  Justification: This threshold corresponds to itemsets that appear in at least ≈4,200 orders, ensuring statistical reliability. This setting yields approximately 300–350 rules, providing enough coverage to effectively analyze co-purchase patterns without introducing overwhelming noise.

- **Minimum Confidence (`min_conf`): 0.05 (or 5%).**

  Justification: A mid-level confidence of 0.05 effectively filters out random associations while preserving interesting, moderately strong ones. This prevents generating recommendations where the conditional probability is too low to be reliable.

This combination of `min_sup` = 0.004 and `min_conf` = 0.05 produces a manageable and meaningful set of strong, interpretable rules for the subsequent recommendation engine.

2.4.1 Conclusion:

The combination of `min_sup = 0.004` and `min_conf = 0.05` represents an optimal trade-off, resulting in a smaller but high-quality set of statistically significant association rules. These rules are ready to be used in the next phase (Part C) for powering the recommendation engine.

2.5 FP-Growth Algorithm (Generalized Steps)

Input:

- Transaction database D
- Minimum support threshold `min_sup`

Output:

- Complete set of frequent itemsets

Algorithm Steps:

1. **Scan the database and find frequent items**
   - Scan the transaction database once.
   - Count the frequency (support count) of each item.
   - Remove infrequent items (those with support < `min_sup`).

- Sort the frequent items in descending order of frequency — this ensures a consistent ordering.

2. **Build the FP-tree (Frequent Pattern Tree)**
   - Create a root node labeled "null."
   - For each transaction in the database:
     - Remove infrequent items.
     - Sort the remaining items according to the global frequency order.
     - Insert the sorted items into the FP-tree:
       * If an item already exists along the current path, increment its count.
       * Otherwise, create a new node.
     - Maintain header links (linked lists connecting same-item nodes).
3. **Mine the FP-tree recursively**
   - Use the divide-and-conquer approach:
   - For each item i in the header table (starting from the least frequent):
     - Generate conditional pattern base:
       * Extract all prefix paths ending with i.
       * Construct conditional FP-tree from the pattern base.
     - If the conditional FP-tree is non-empty:
       * Recursively mine it to find frequent patterns that include i.
     - Combine i with patterns found in its conditional tree to form larger frequent itemsets.
4. **Output**
   - The union of all frequent itemsets found in the recursive steps forms the complete set of frequent patterns.

3 Part C: Recommendation and Evaluation

This phase validates the utility of the association rules mined in Part B by deploying them within a simulated recommendation engine and evaluating their performance against the reserved Test Set.

3.1 Rule Lists and Common Rules

The complete set of association rules generated using the optimal thresholds (`min_sup` = 0.004, `min_conf` = 0.05) was ranked to produce two lists containing the top 100 rules: one sorted strictly by Support and the other by Confidence.

Generation: These lists were saved to separate files (`5_top100RulesBySup.txt` and `5_top100RulesByConf.txt`). The common rules were identified by finding the intersection of the two sets based on the `(antecedent, consequent)` pairs.

Intersection: The analysis revealed **38** common rules appearing in both top 100 lists. These common rules are highly valuable, as they represent patterns that are both frequently observed (high support) and reliable predictors (high confidence), making them the most actionable recommendations for immediate business deployment.

Qualitative Inferences from Top Rules: Analysis of these common rules, along with the top 100 lists, revealed three primary themes indicative of strong customer behavior:

- **Organic Loyalty:** The most dominant pattern is the cross-purchase of different organic products. A significant number of rules linked one organic fruit/vegetable (e.g., *Organic Raspberries*) to another (e.g., *Organic Strawberries*). This suggests that once a customer commits to the *Organic* attribute, they are highly likely to maintain that preference across related product categories.

- **Staple Item Anchors:** The most frequent consequents (the predicted items) in the top rules are high-frequency staples, particularly *Banana* and *Bag of Organic Bananas*. This confirms that these items act as anchors in grocery baskets, often driving subsequent, less frequent purchases.

- **Cross-Category Complements:** Rules were observed linking one category to an expected complementary one, such as produce items (e.g., *Organic Whole Milk* $\rightarrow$ *Organic Strawberries*) or primary ingredients (e.g., *Limes* $\rightarrow$ *Large Lemon*), highlighting functional co-purchase behavior.

3.2 Implementation of Evaluation Logic

To rigorously test the predictive capability of the rules, a realistic simulation of a shopping session was created using the 20% Test Set.

Transaction Splitting:

- For every transaction (basket) with two or more items, the transaction was divided into:
  - **Input:** First half of the items (simulating the user's current cart).
  - **Ground Truth:** Second half of the items (the target prediction set).

Recommendation Generation:

- The rule dictionary was queried using each product in the "Input" set.
- All consequent products (Y) from rules of the form $\{A\} \rightarrow Y$ were aggregated to form the recommendation set.

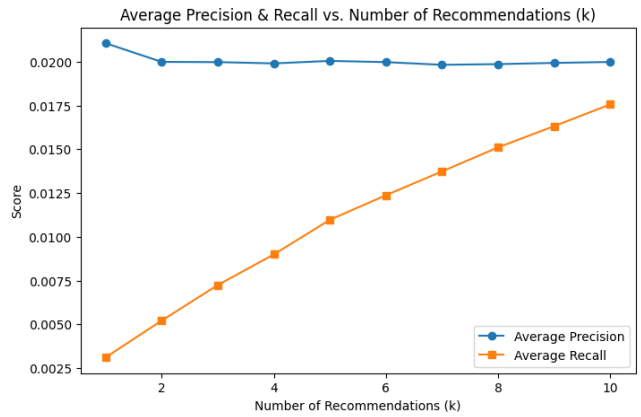

Figure 4: Average Precision and Recall vs. Number of Recommendations (k)

3.3 Calculation and Plotting of Average Metrics

The recommender's performance was evaluated using standard metrics—**Precision** and **Recall**—as the number of recommendations (k) was varied from 1 to 10.

Analysis of Average Metrics:

- **Average Precision:** Starts at approximately 0.021 for $k = 1$ and remains relatively flat around 0.020 up to $k = 10$.

  Interpretation: This low but stable precision indicates that the majority of recommended items are not part of the user's ground truth, which is common in sparse grocery datasets with high basket diversity.

- **Average Recall:** Starts very low at ≈ 0.003 and increases nearly linearly to ≈ 0.0175 at $k = 10$.

  Interpretation: As k increases, recall improves, showing the classic trade-off between precision and recall. The flat precision suggests diminishing returns at higher k values.

3.4 Sample Analysis

To illustrate the model's performance across user types, the Precision and Recall curves for a few sample users were analyzed.

Basket 751,19146,10074

These graphs show both precision and recall are zero across all values of k (number of recommendations). This indicates that the FP-growth algorithm failed to generate any relevant recommendations for this Basket. This is a common issue in association rule-based systems, likely caused by a cold-start problem or a very sparse transaction history. The Basket's purchasing patterns

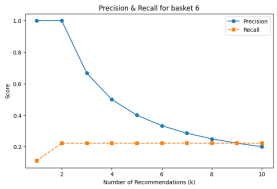

Figure 5: Basket 6

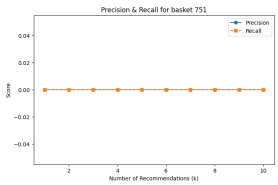

Figure 6: Basket 751

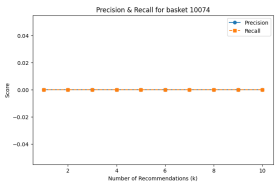

Figure 7: Basket 10074

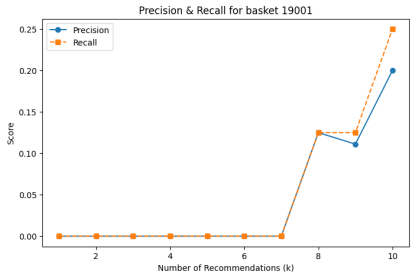

Figure 8: Basket 19001

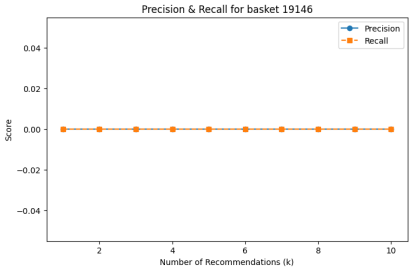

Figure 9: Basket 19146

Figure 10: Precision and Recall curves for sample Baskets

might not meet the minimum support threshold necessary to form frequent itemsets, and consequently no association rules could be generated.

Basket 6

The results for Basket 6 are quite promising. Precision starts at a perfect 1.0 for $k = 1$ and $k = 2$, meaning every recommendation in the top two was relevant, demonstrating high accuracy for the most prioritized suggestions. As k increases, precision gradually decreases, which is expected behavior. Recall starts low but quickly stabilizes around 0.22 for k values from 2 to 10. This suggests the model successfully identifies about 22% of all relevant items within the top recommendations. The stabilization indicates that most easily discoverable relevant items have been found, and providing more recommendations beyond this does not significantly improve coverage.

Basket 19001

Basket 19001 presents an interesting case where both precision and recall remain zero from $k = 1$ to $k = 7$, indicating initial failure to find suitable recommendations. However, at $k = 8$, there is a sharp simultaneous increase: recall peaks at 0.25 and precision reaches 0.20 by $k = 10$. This pattern suggests that relevant items are associated with less frequent or more specific patterns, which only emerge when a larger recommendation list is considered. The algorithm uncovers niche or subtle relationships later in the ranking, highlighting that some Baskets require more extensive recommendations to receive relevant items.

3.5 Conclusion and Future Work

The evaluation demonstrates that Market Basket Analysis successfully uncovered actionable patterns, especially for highly predictable Baskets (as seen in Case 1). However, the low average precision underscores the limitations of rule-based systems in sparse datasets with diverse baskets.

Future Work:

- Integrate association rules with hybrid recommenders (e.g., collaborative filtering, matrix factorization).
- Experiment with lower `min_sup` thresholds in combination with rule pruning for better coverage.
- Address cold-start Baskets through demographic or session-based features.